

Sistema de Banco de Dados do RadarTorres

Persistência de dados do protótipo de detecção, localização e seleção automática de ação por sensoramento

1. Visão geral

O RadarTorres é uma aplicação desktop desenvolvida em C#/WPF (.NET 9) que se comunica com um Arduino por porta serial, recebe leituras de sensores (ângulo e distância), exibe os alvos em um radar de quatro quadrantes e seleciona automaticamente a torre mais adequada para cada alvo. Para que o sistema seja auditável e multiusuário, é necessário guardar de forma permanente quem acessa a aplicação, o que foi detectado, quais ações foram executadas e como os modos de operação foram alterados. Essa é a função do sistema de banco de dados apresentado neste documento.

Na etapa atual do projeto, a persistência é feita em **arquivos CSV**, uma decisão tomada deliberadamente para manter a solução simples e totalmente explicável. Cada arquivo funciona como uma tabela: a primeira linha é o cabeçalho com o nome das colunas e cada linha seguinte é um registro. Toda a camada de dados foi construída de forma que a troca futura por um banco relacional (SQLite ou SQL Server) não exija mudanças no restante da aplicação.

2. Para que o banco de dados é usado

O armazenamento cumpre quatro finalidades no sistema:

- **Controle de acesso:** guarda as contas de usuário, com senha protegida por hash e o perfil de permissão (Administrador, Operador ou Visualizador), permitindo o login independente da conta do Windows.
- **Histórico de detecções:** registra cada alvo na primeira vez em que ele é detectado, com posição, quadrante, dispositivo de origem e data/hora.
- **Auditoria de ações e de modos:** registra todo acionamento demonstrativo (manual ou automático, executado, cancelado ou com erro) e toda troca de modo de operação da torre, permitindo reconstituir o que aconteceu em uma sessão.
- **Preferências do usuário:** guarda idioma, tema, estado da barra lateral, tela inicial e quantidade de registros por página de cada usuário, para que a aplicação abra sempre do jeito que ele configurou.

3. Onde os dados ficam

Os arquivos são gravados na pasta %AppData%\RadarTorres\Data\, que pertence ao usuário do Windows. Essa escolha evita a pasta de instalação (C:\Program Files), que é somente leitura para usuários comuns, faz a aplicação funcionar igualmente instalada ou em desenvolvimento e preserva os dados em caso de desinstalação e reinstalação. São cinco arquivos: usuarios.csv, objetos_detectados.csv, acoes_realizadas.csv, modo_atual_torre.csv e preferencias_usuario.csv.

4. Modelo físico

A Figura 1 apresenta o modelo físico dos dados. As tabelas **objetos_detectados** e **modo_atual_torre** são independentes. A tabela **preferencias_usuario** referencia o *Id* de **usuarios** (um usuário tem no máximo uma linha de preferências), e **acoes_realizadas** referencia o *Login* do usuário responsável, campo que fica vazio nas ações automáticas. As linhas tracejadas indicam referências lógicas: como o armazenamento é CSV, a consistência é garantida pelo código, e não pelo arquivo.



Figura 1. Modelo físico do banco de dados do RadarTorres (dos autores)

Tabela 1. Resumo das tabelas

Tabela	Conteúdo	Operações
usuarios	Contas de acesso: nome, login, hash e salt da senha, perfil, situação e datas de criação e último acesso.	Consulta, inclusão e atualização
objetos_detectados	Primeira detecção de cada alvo: tipo, coordenadas X/Y/Z, quadrante, data/hora, dispositivo e nível de confiança.	Somente inclusão e consulta
acoes_realizadas	Cada tentativa de acionamento: dispositivo, tipo, posição, usuário, origem e resultado.	Somente inclusão e consulta
modo_atual_torre	Cada troca de modo: modo anterior, novo modo, horários de solicitação e execução e resultado.	Somente inclusão e consulta
preferencias_usuario	Idioma, tema, barra lateral, tela inicial e registros por página de cada usuário.	Consulta e gravação por usuário

5. Como o banco de dados é usado

O acesso aos dados segue o padrão **Repositório**. Cada tabela possui uma interface própria (por exemplo, `IUsuarioRepository` e `IAcaoRealizadaRepository`) e uma implementação em CSV (`CsvUsuarioRepository` e assim por diante). As implementações são registradas no contêiner de injeção de dependência em `App.xaml.cs`, de modo que ViewModels e serviços dependem apenas da interface e nunca sabem como os dados são gravados.

Todas as implementações usam uma classe genérica, `CsvTableStore`, que concentra a leitura e a escrita dos arquivos. Seu funcionamento pode ser resumido assim:

- **Criação automática:** na primeira vez que uma tabela é usada, o arquivo é criado com a linha de cabeçalho. O próprio código faz o papel da migração: adicionar uma coluna significa adicionar uma propriedade ao modelo e uma coluna ao repositório.
- **Inclusão rápida:** novos registros são acrescentados ao final do arquivo, sem reescrever a tabela inteira. É assim que são gravadas as detecções, as ações e as trocas de modo.
- **Atualização:** quando um registro precisa mudar (por exemplo, editar um usuário), a tabela é reescrita por completo, o que é aceitável no volume de dados do projeto.
- **Concorrência e formato:** cada operação é protegida por trava (*lock*), os arquivos usam codificação UTF-8 e campos com vírgula ou aspas são escapados conforme o padrão CSV.

Na prática, os registros surgem do uso normal da aplicação: o login consulta **usuarios** e atualiza o último acesso; a primeira detecção de um alvo gera uma linha em **objetos_detectados**; toda tentativa de disparo passa pelo serviço de acionamento, que grava em **acoes_realizadas**; e toda troca de modo, confirmada ou cancelada, é registrada em **modo_atual_torre**. As tabelas de auditoria não oferecem alteração nem exclusão, o que preserva a integridade do histórico. No primeiro uso, um usuário administrador padrão é criado automaticamente, e sua senha deve ser trocada.

6. Vantagens da abordagem adotada

- **Simplicidade:** não há servidor de banco para instalar ou configurar, e o instalador do sistema continua autossuficiente.
- **Transparência:** os arquivos podem ser abertos em qualquer editor ou planilha, o que facilita a conferência dos dados e a demonstração na defesa do TCC.
- **Segurança das senhas:** as senhas nunca são gravadas em texto puro; usa-se PBKDF2-HMACSHA256 com 100.000 iterações e um salt de 128 bits por usuário.
- **Rastreabilidade:** as tabelas de auditoria somente de inclusão formam um histórico confiável de detecções, ações e modos.
- **Preparado para evoluir:** graças às interfaces de repositório, a migração para SQLite ou SQL Server consiste em criar novas implementações e trocar o registro na injeção de dependência, sem alterar telas, ViewModels ou serviços.

7. Limitações e evolução

Por ser baseado em CSV, o armazenamento atual não impõe chaves estrangeiras nem índices, e as atualizações reescrevem o arquivo inteiro, o que limita o desempenho com grandes volumes de dados. Esses pontos estão sinalizados no código com a marcação `TODO(SQL)` e serão resolvidos na migração planejada para um banco relacional, quando as referências lógicas da Figura 1 passarão a ser chaves estrangeiras reais.